Digital Electronics (EC2L004)

Lecture 19: Crash Course on Verilog – Part I

What is Verilog HDL?

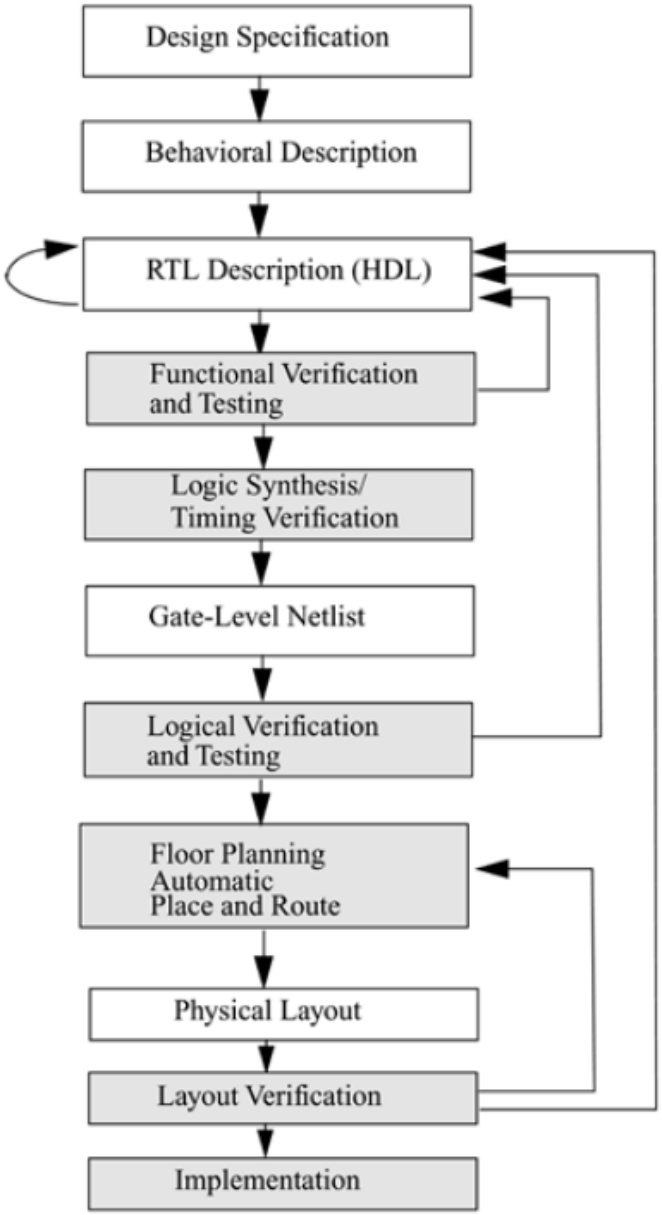
- A high-level language that can model, represent, and simulate the digital circuits.
  - Hardware concurrency
  - Parallel activity flow
  - Different levels of abstraction to be mixed in the same model

- Design examples using Verilog HDL
  - Intel Pentium, ARM7, and many more.

- VHDL represents another high-level language for digital system design.

- In this course, we study Verilog HDL
  - Reason: Most widely used in the industry and easy to learn, syntax is similar to C

Typical Design Flow of ICs



Note: Shaded blocks denote the processes in the design flow whereas unshaded blocks show the level of design representation.

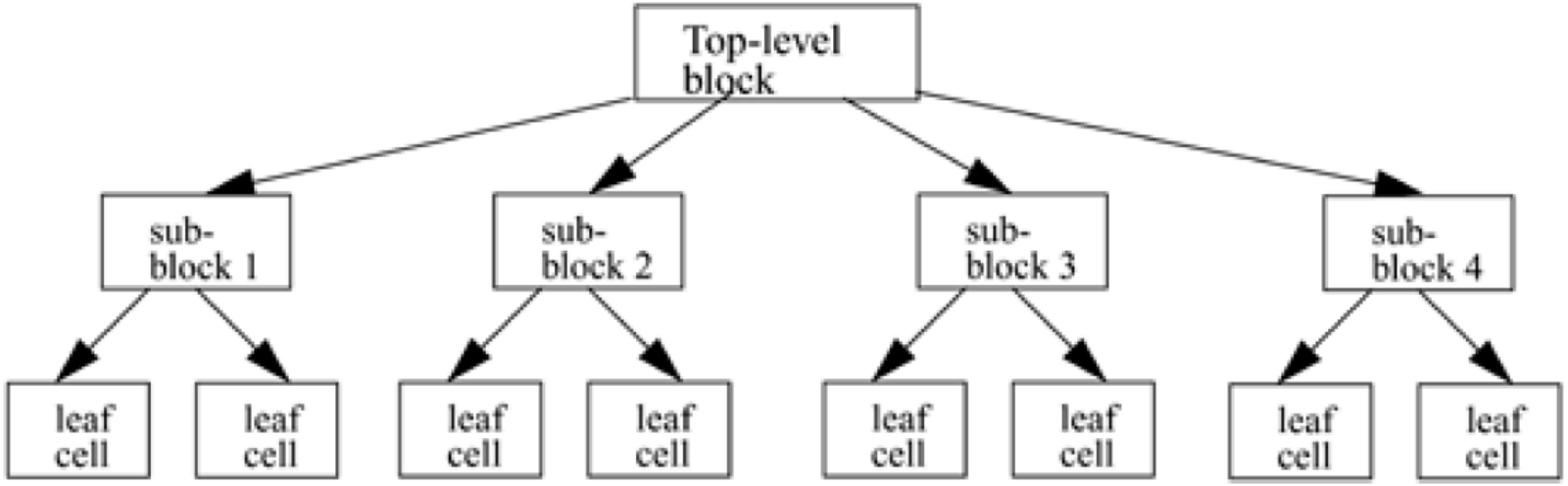
Importance of HDLs

- Designs can be described at a very abstract level using an HDL without considering the fabrication details
  - Logic synthesis tools create the netlist which is dependent on the technology.
- Functional verification can be done in the early design cycle.
  - Reduced design cycles
- Designing with HDL is like writing computer programs with comments which can provide a concise representation of the design, compared to a gate-level schematics.

Hierarchical Modeling Concepts

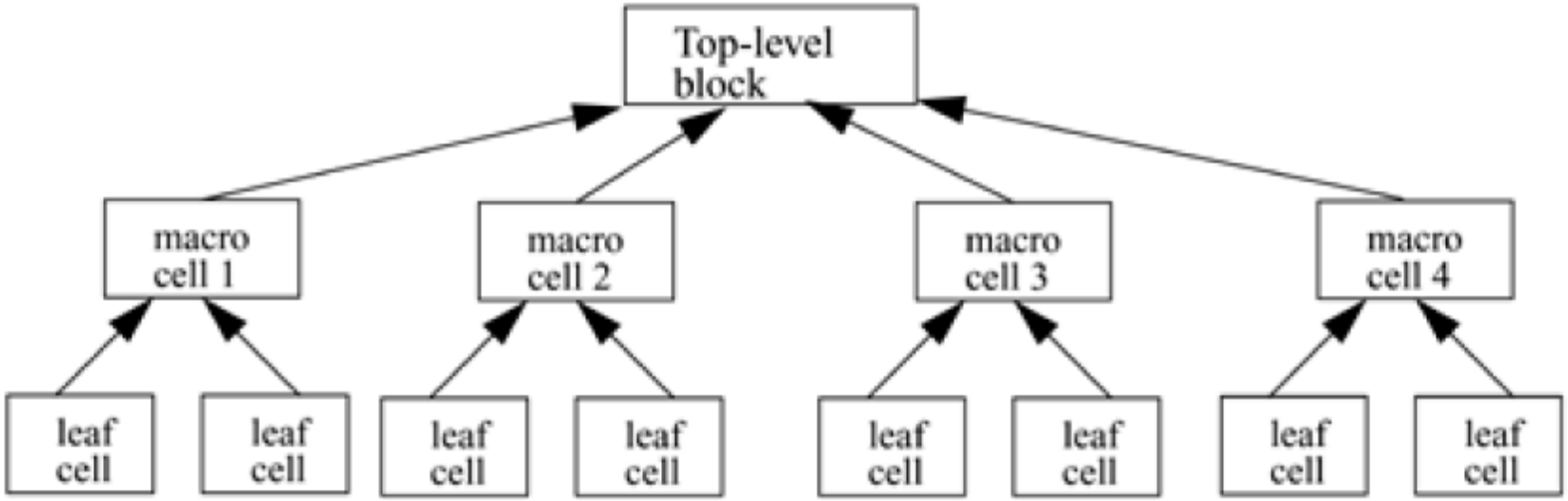
Design Methodologies

- Top-down approach



Design Methodologies

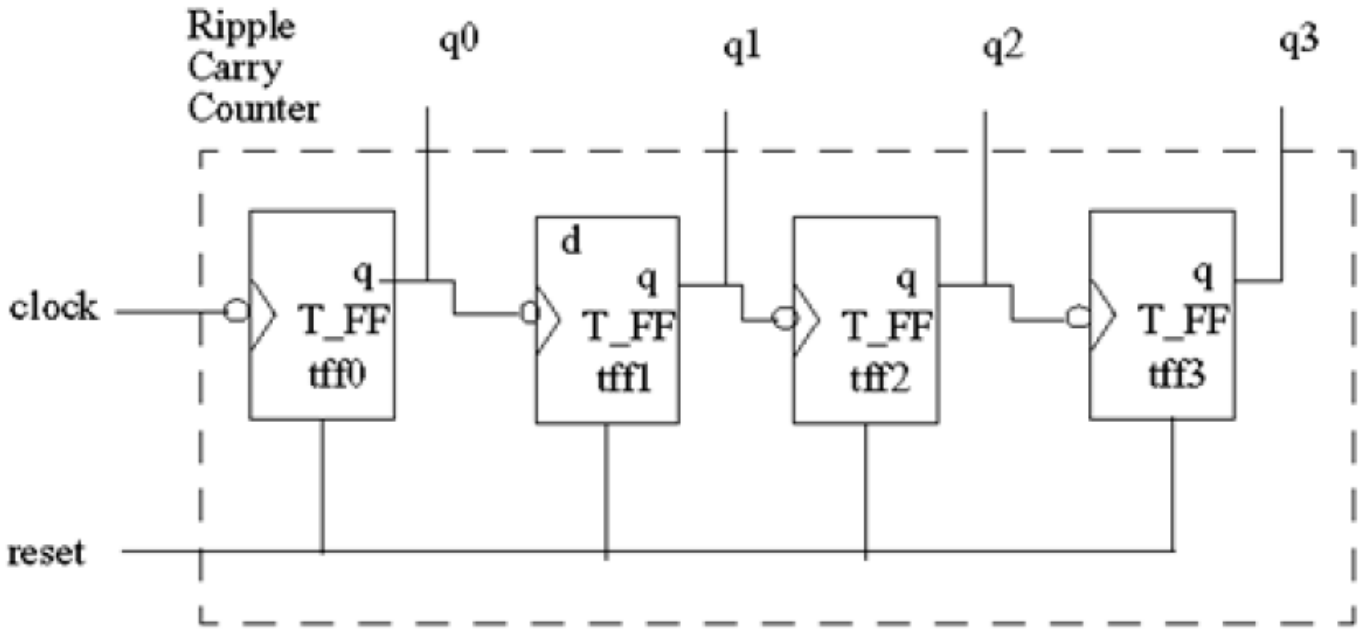
- Bottom-Up approach

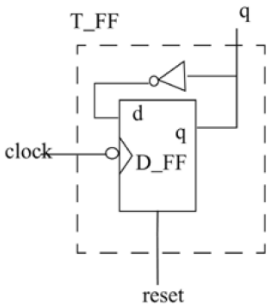


Typically, a combination of top-down and bottom-up flow is used.

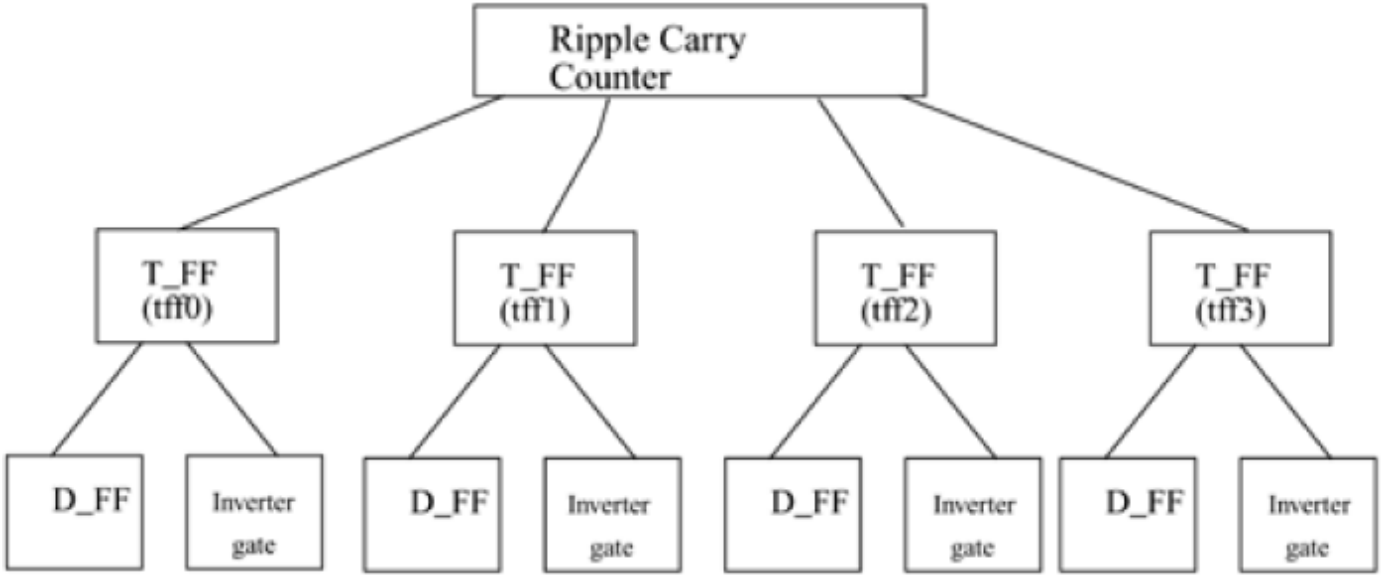
Example:

- 4-bit Ripple Carry Counter





Ripple Carry Counter: Design Hierarchy



Modules

- Module is the basic building block in Verilog.
  - Element or collection of lower-level design blocks
  - Communicates to the higher level blocks through its port interface (inputs and outputs)
  - Hides the internal implementation details
    - Allows the designer to modify the module internals without affecting the rest of the design.

```
module <module_name> (<module_terminal_list>);
...
<module internals>
...
...
endmodule
```

```
module T_FF (q, clock, reset);
.
.
<functionality of T-flipflop>
.
.
endmodule
```

Modules (Cont'd)

- Internals of the module can be defined at four levels of abstraction
  - Behavioral or algorithmic level
    - Similar to C program
    - Highest level of abstraction
  - Dataflow level
    - Designed according to the flow of data between registers and how the data is processed in the design
  - Gate level
    - Similar to describing a design in terms of a gate level logic diagram
  - Switch level
    - Implemented in terms of switches, storage nodes, and the interconnections between them.
    - Lowest level of abstraction
- Verilog allows to mix and match all four levels of abstractions in a design.

Example: Levels of abstraction-Behavioural

```
module mux2_to_1 (out, s, i0, i1);
    // Port declarations from the I/O diagram
    output out;
    input i0, i1;
    input s;
    reg out;

    always @(s or i0 or i1)
        case (s)
            1'd0 : out = i0;
            1'd1 : out = i1;
            default: $display("Invalid control signals");
        endcase
endmodule
```

Example: Levels of abstraction-Data flow

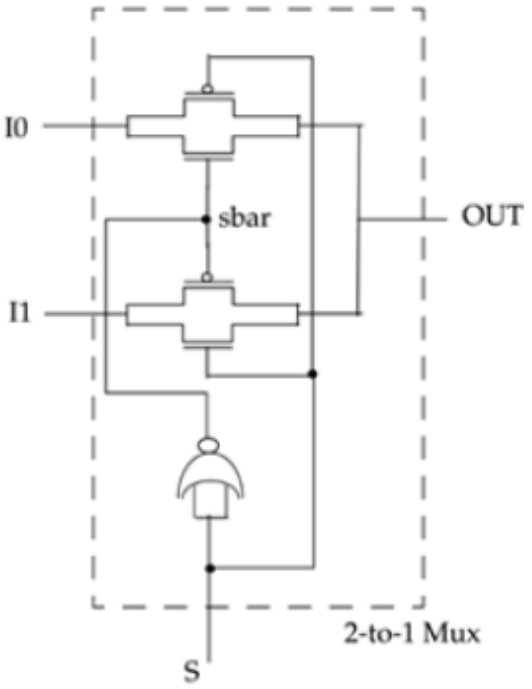
```
module mux2_to_1 (out, s, i0, i1);
    // Port declarations
    output out;
    input i0, i1;
    input s;

    assign out = s ? i1: i0;
endmodule
```

Example: Levels of abstraction-Gate Level

```
module mux2_to_1 (out, s, i0, i1);
    output out;
    input i0, i1;
    input s;

     wire y0, y1, y2, y3;// Internal wire declarations
    // Gate instantiations
    // Create sn signal.

    not (sn, s);

    // 3-input and gates instantiated
    and (y0, i0, sn);
    and (y1, i1, s);

    //2-input or gate instantiated
    or (out, y0, y1);

endmodule
```

Example: Levels of abstraction-Switch Level

```
module mux2_to_1 (out, s, i0, i1);

    output out;
    input s, i0, i1;

    //internal wire
    wire sbar; //complement of s

    //equivalent to a not gate
    my_nor nt(sbar, s, s);

    //instantiate cmos switches
    cmos (out, i0, sbar, s);
    cmos (out, i1, s, sbar);

endmodule
```



Instances

- Module is just a template from which actual objects are created.
  - Specify how the module will work, and its interface; modules must be instantiated for use in the design.

- Unique object for each invocation of module

- Each object has its own---name, Variables, parameters, and I/O interface

- *Instantiation* is nothing but the process of creating objects from a module template, and the objects are called “instances”.

Example: Instantiation

```
// Define the top-level module called ripple carry
// counter. It instantiates 4 T-flipflops.
```

```
module ripple_carry_counter(q, clk, reset);

    output [3:0] q;
    input clk, reset;

    // Four instances of the module T_FF are created.
    // Each has a unique name.
    // Each instance is passed a set of signals.
    // Notice, that each instance is a copy of the module T_FF.

    T_FF tff0(q[0],clk, reset);
    T_FF tff1(q[1],q[0], reset);
    T_FF tff2(q[2],q[1], reset);
    T_FF tff3(q[3],q[2], reset);
endmodule
```

Example: Instantiation

//Define the module T_FF. It instantiates a D-flipflop. We assumed that module D-flipflop is defined elsewhere in the design.

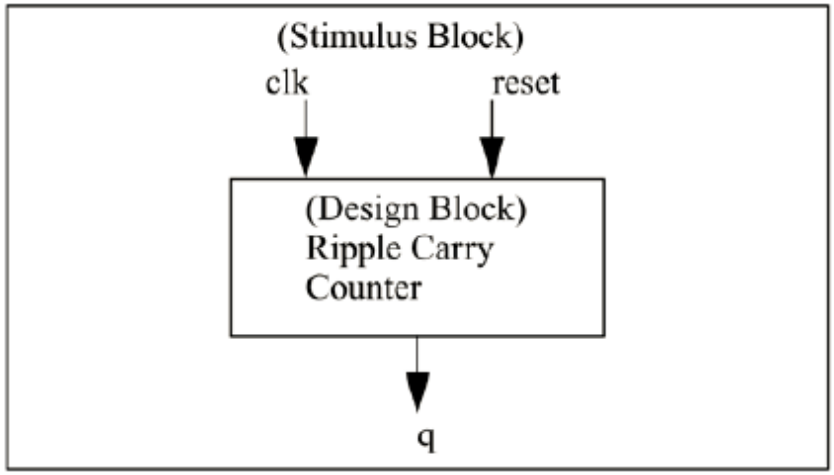
```
module T_FF(q, clk, reset);
    output q;
    input clk, reset;
    wire d;

    // Instantiate D_FF. Call it dff0.
    D_FF dff0(q, d, clk, reset);

    // not gate is a Verilog primitive.
    not n1(d, q);

endmodule
```

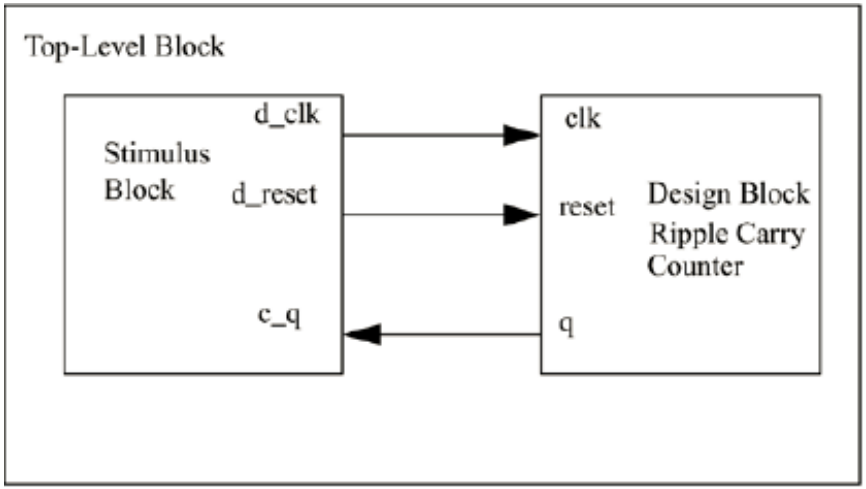
Example: D Flip-Flop

```
// module D_FF with synchronous reset
module D_FF(q, d, clk, reset);
    output q;
    input d, clk, reset;
    reg q;
    //Lots of new constructs. Ignore the functionality
    // of the constructs for now. Concentrate on how the
    // design block is built in a top-down fashion.

    always @(posedge reset or negedge clk)
        if (reset)
            q <= 1'b0;
        else
            q <= d;
endmodule
```

Components of a Simulation

- Functionality of the design has to be tested by applying stimulus and checking the results
  - Keep the stimulus and design block always separate.
- Verilog itself can be used to write the stimulus
  - Stimulus block is also often called as “test bench”
- Two styles of stimulus applications are possible

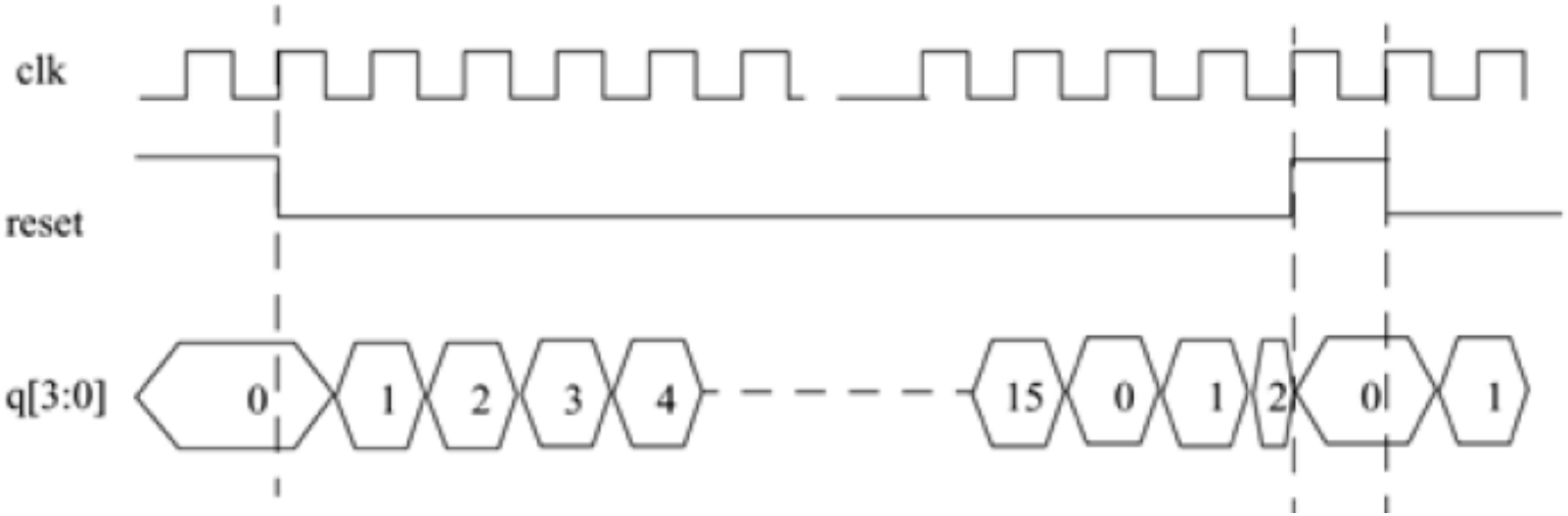




Example: Test bench

```
module stimulus;
    reg clk;
    reg reset;
    wire[3:0] q;

    // instantiate the design block
    ripple_carry_counter r1(q, clk, reset);
    // Control the clk signal that drives the design block.
    //Cycle time = 10

    initial
        clk = 1'b0; //set clk to 0
    always
        #5 clk = ~clk;//toggle clk every5 time units

    // Control the reset signal that drives the design block
    // reset is asserted from 0 to 20 and from 200 to 220.
    initial
    begin
        reset = 1'b1;
        #15 reset = 1'b0;
        #180 reset = 1'b1;
        #10 reset = 1'b0;
        #20 $finish; //terminate the simulation
    end
    // Monitor the outputs
    initial
    $monitor($time, " Output q = %d", q);
endmodule
```

Example: Counter Simulation Output



Example: Counter Simulation Output

```
  0 Output q =  0
 20 Output q =  1
 30 Output q =  2
 40 Output q =  3
 50 Output q =  4
 60 Output q =  5
 70 Output q =  6
 80 Output q =  7
 90 Output q =  8
100 Output q =  9
110 Output q = 10
120 Output q = 11
130 Output q = 12
140 Output q = 13
150 Output q = 14
160 Output q = 15
170 Output q =  0
180 Output q =  1
190 Output q =  2
195 Output q =  0
210 Output q =  1
220 Output q =  2
```

Basic Concepts, Modules, and Ports

Learning Objectives

- Define the logic value set and data types such as
  - nets,
  - registers,
  - vectors,
  - Numbers

- Identify useful system tasks for displaying and monitoring information, and for stopping and finishing the simulation.

Operators and Number Specification

- Operators
  - Unary, binary and ternary
  - A = ~b; a = b&&c; a = b?c : d;
- Number specification
  - Sized numbers: <size>’<base format><number>
  - <size> -number of bits
  - <size> is specified only in decimal
  - 4’b1111; 12’habc

Keywords

- Keywords are special identifiers reserved to define the language constructs.
  - Keywords are in lowercase.

```
reg value;      // reg is a keyword; value is an identifier
input clk;      // input is a keyword, clk is an identifier
```

Identifiers

- Identifiers are names given to objects so that they can be referenced in the design.
  - Identifiers made of alphanumeric characters, the underscore (_) and the dollar ($) sign and starts with alphabets or underscore.
  - Identifiers are case sensitive.
  - The dollar sign as first character is reserved for system tasks.
    - Ex : $monitor

Data Types

Value Set

- Verilog support four values and eight strengths to model the functionality of real hardware.

- The four value levels are:

Value Level	Condition in Hardware Circuits
0	Logic zero, false condition
1	Logic one, true condition
x	Unknown logic value
z	High impedance, floating state

Nets

- Represent connection between hardware elements.

- Declared by keyword **wire,** by default one-bit.
  - wire a;
  - wire b, c;
  - wire d = 1’b0;
  - Wire and net often used interchangeably.

Registers

- Represent data storage elements
  - Retain value until another value is placed onto them.
  - Do not confuse the term registers in Verilog with hardware registers built from edge-triggered flip-flops in real circuits.
- A variable that stores value
  - Unlike a net, it needs no driver
- Default value for a ‘reg’ variable is x
- Declared using keyword **reg**

Registers

```
reg reset;        // declare a variable reset that can hold its value
initial           // this construct will be discussed later
   reset = 1'b1; //initialize reset to 1 to reset the digital circuit.
   #100 reset = 1'b0;  // after 100 time units reset is deasserted.
end
```

Vectors

- Nets or reg data types can be declared as vectors (multiple bit widths). If bit width is not specified, the default is scalar (1-bit).
  - wire [7:0] busA, busB;
  - reg [16:0] address;
- [high#:low#] or [low#:high#], but the most significant bit is the left number in the square bracket
  - reg [0:40] addr1; wire [15:12] addr2

System Tasks

System Tasks

- Verilog provides standard system tasks for certain routine operations.
  - All system tasks appear in the form $<keyword>.
  - Operations such as displaying on the screen, monitoring values of nets, stopping, and finishing are done by system tasks.

System Tasks …cont'd

- $display
  - Like printf and used for displaying values of variables and expressions
  - Values can be printed in decimal (%d), binary (%b), string (%s), hex (%h), ASCII character (%c), octal (%o), real in scientific (%e), real in decimal (%f), real in scientific or decimal whichever is shorter(%g).
  - A $display inserts a newline at the end of the string by default.
  - A $display without any arguments produces a newline.

$display

```
//Display value of 41-bit virtual address 1fe0000001c at time 200
reg [0:40] virtual_addr;
$display("At time %d virtual address is %h", $time, virtual_addr);
-- At time 200 virtual address is 1fe0000001c
//Display value of port_id 5 in binary
reg [4:0] port_id;
$display("ID of the port is %b", port_id);
-- ID of the port is 00101
//Display x characters
//Display value of 4-bit bus 10xx (signal contention) in binary
reg [3:0] bus;
$display("Bus value is %b", bus);
-- Bus value is 10xx
//Display the hierarchical name of instance p1 instantiated under
//the highest-level module called top. No argument is required. This
//is a useful feature)
$display("This string is displayed from %m level of hierarchy");
-- This string is displayed from top.p1 level of hierarchy
```

System Tasks

- $monitor
  - Monitor a signal when its value changes.
  - Only one active monitor list.
  - Format like the $display task is used.

- If more than one then, the last one statement will be the active one.

- Monitoring turned ON by default at start of simulation and can be controlled during the simulation using $monitoron and $monitoroff.

$monitor

```
//Monitor time and value of the signals clock and reset
//Clock toggles every 5 time units and reset goes down at 10 time units
initial
begin
    $monitor($time, " Value of signals clock = %b reset = %b", clock, reset);
end

Partial output of the monitor statement:
-- 0 Value of signals clock = 0 reset = 1
-- 5 Value of signals clock = 1 reset = 1
-- 10 Value of signals clock = 0 reset = 0
```

System Tasks

- $stop – stops simulation and puts it into interactive mode
- $finish – terminates the simulation

```
// Stop at time 100 in the simulation and examine the results
// Finish the simulation at time 1000.
initial // to be explained later. time = 0
begin
clock = 0;
reset = 1;
#100 $stop; // This will suspend the simulation at time = 100
#900 $finish; // This will terminate the simulation at time = 1000
end
```

Modules and Ports

Learning Objectives

- Identify the components of a Verilog module definition
- Understand how to define the port list for a module and declare it in Verilog.
- Describe the port connection rules in a module instantiation.
- Understand how to connect ports to external signals, by ordered list, and by name.

Components of a Verilog Module

Module Name,
Port List, Port Declarations (if ports present)
Parameters (optional),

Declarations of **wires**, **regs** and other variables

Data flow statements (**assign**)

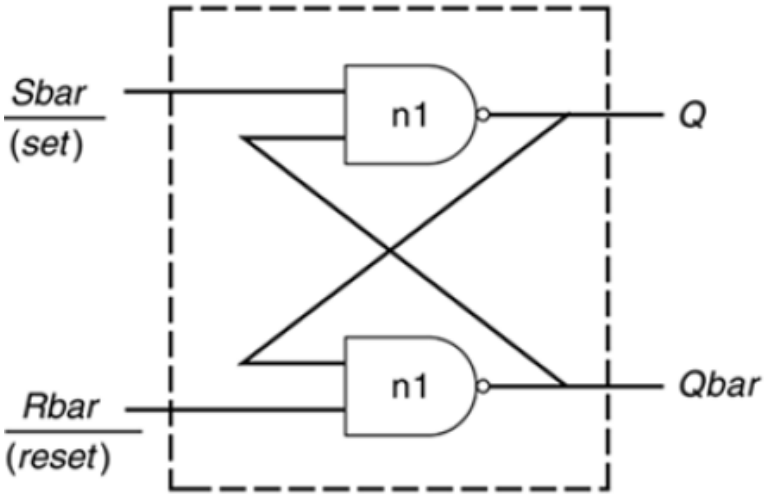
Instantiation of lower level modules

always and **initial** blocks. All behavioral statements go in these blocks.

Tasks and functions

endmodule statement

Modules (Contd.)




```
// Module name and port list
// SR_latch module
module SR_latch(Q, Qbar, Sbar, Rbar);
//Port declarations
output Q, Qbar;
input Sbar, Rbar;
// Instantiate lower-level modules
// In this case, instantiate Verilog primitive nand gates
// Note, how the wires are connected in a cross-coupled fashion.
nand n1(Q, Sbar, Qbar);
nand n2(Qbar, Rbar, Q);
// endmodule statement
endmodule
```

Modules (Contd.)

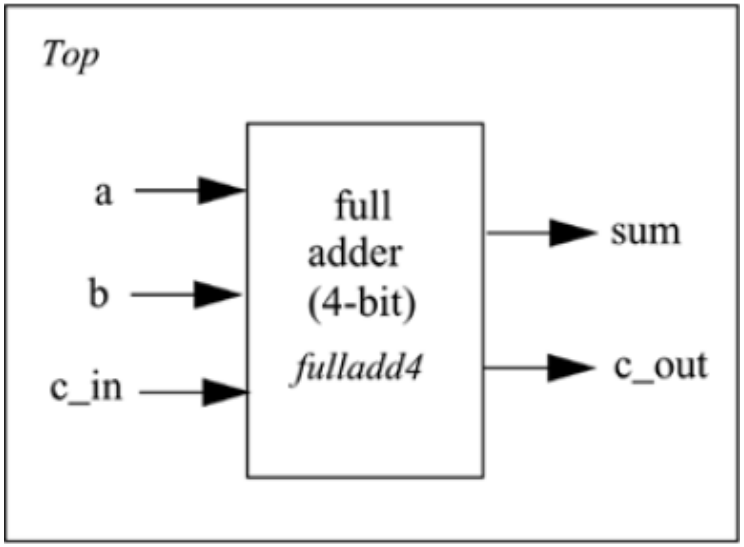
```
// Module name and port list
// Stimulus module
module Top;
// Declarations of wire, reg, and other variables
wire q, qbar;
reg set, reset;
// Instantiate lower-level modules
// In this case, instantiate SR_latch
// Feed inverted set and reset signals to the SR latch
SR_latch m1(q, qbar, ~set, ~reset);
// Behavioral block, initial
initial
begin
$monitor($time, " set = %b, reset= %b, q= %b\n",set,reset,q);
set = 0; reset = 0;
#5 reset = 1;
#5 reset = 0;
#5 set = 1;
end
// endmodule statement
endmodule
```

Ports

- Ports provide the interface by which a module can communicate with its environment.
- The environment can interact with the module only through its ports.
- The internals of the module are not visible to the environment.
- Ports are also referred to as “terminals”

List of ports

- A module definition contains an optional list of ports.
  - If the module does not exchange any signals with the environment, there are no ports in the list.




```
module fulladd4(sum, c_out, a, b, c_in);          //Module with a list of ports
module Top;                  // No list of ports, top-level module in simulation
```

Port Declaration

- All ports in the list of ports must be declared in the module.
- Port declarations –input, output, inout
  - Port declarations are implicitly declared as **wire**

```
module fulladd4(sum, c_out, a, b, c_in);
//Begin port declarations section
output[3:0] sum;
output c_cout;
input [3:0] a, b;
input c_in;
//End port declarations section
...
<module internals>
...
endmodule
```

Port Declaration (Contd.)

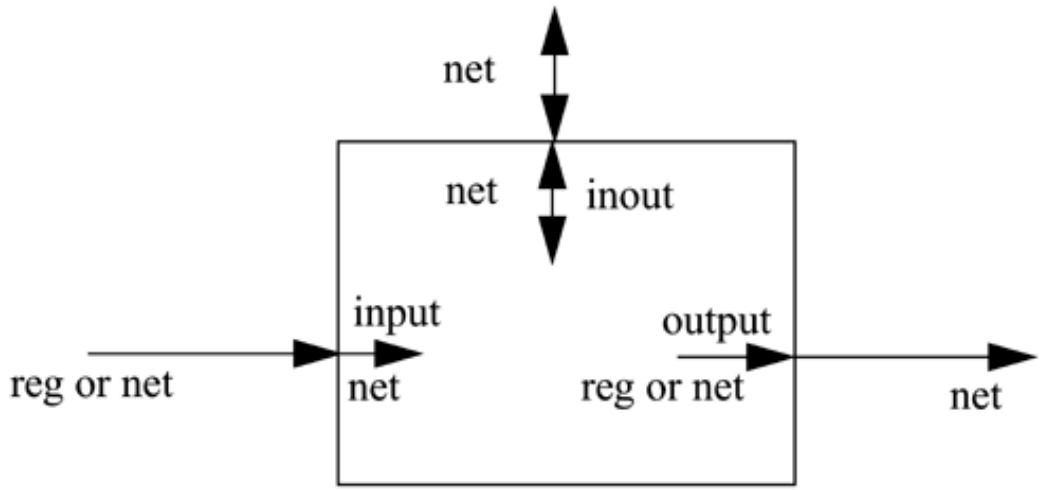
- Port declarations for DFF

```
module DFF(q, d, clk, reset);
output q;
reg q; // Output port q holds value; therefore it is declared as reg.
input d, clk, reset;
...
...
endmodule
```

- Ports of the type input and inout cannot be declared as reg because reg variables store values and input ports should not store values but simply reflect the changes in the external signals they are connected to.

Port Connection Rules

- One can visualize a port as consisting of two units, one unit that is internal to the module and another that is external to the module.
  - The internal and external units are connected.
  - There are rules governing port connections when modules are instantiated within other modules.



Port Connection Rules (Contd.)

- Unconnected ports
  - Verilog allows ports to remain unconnected.

```
fulladd4 fa0(SUM, , A, B, C_IN);

 // Output port c_out is unconnected
```

Port Connection Rules (Contd.)

- Example of illegal port connection

```
module Top;
//Declare connection variables
reg [3:0]A,B;
reg C_IN;
reg [3:0] SUM;
wire C_OUT;
//Instantiate fulladd4, call it fa0
fulladd4 fa0(SUM, C_OUT, A, B, C_IN);
//Illegal connection because output port sum in module fulladd4
//is connected to a register variable SUM in module Top.
.
.
<stimulus>
.
.
endmodule
```

Connecting Ports to External Signals

- Connecting by ordered list
  - Connecting by ordered list is the most intuitive method for most beginners.
  - The signals to be connected must appear in the module instantiation in the same order as the ports in the port list in the module definition.
- Connecting ports by name
  - For large designs where modules have, say, 50 ports, remembering the order of the ports in the module definition is impractical and error-prone.
  - Verilog provides the capability to connect external signals to ports by the port names, rather than by position.

Connection by Ordered List

```
module Top;
//Declare connection variables
reg [3:0]A,B;
reg C_IN;
wire [3:0] SUM;
wire C_OUT;
//Instantiate fulladd4, call it fa_ordered.
//Signals are connected to ports in order (by position)
fulladd4 fa_ordered(SUM, C_OUT, A, B, C_IN);
...
<stimulus>
...
endmodule

module fulladd4(sum, c_out, a, b, c_in);
output[3:0] sum;
output c_cout;
input [3:0] a, b;
input c_in;
...
<module internals>
...
endmodule
```

Connecting port by name

```
// Instantiate module fa_byname and connect signals to ports by name
fulladd4 fa_byname(.c_out(C_OUT), .sum(SUM), .b(B), .c_in(C_IN),
.a(A),);

// Instantiate module fa_byname and connect signals to ports by name
fulladd4 fa_byname(.sum(SUM), .b(B), .c_in(C_IN), .a(A),);
```

- Another advantage of connecting ports by name is that as long as the port name is not changed, the order of ports in the port list of a module can be rearranged without changing the port connections in module instantiations.

Gate-Level Modeling

Learning Objectives

- Identify logic gate primitives provided in Verilog.

- Understand how to construct a Verilog description from the logic diagram of the circuit.

Gate Types

- Verilog supports basic logic gates as predefined primitives.
  - These primitives are instantiated like modules except that they are predefined in Verilog and do not need a module definition.

Gate Types

- There are two classes of basic gates: *and/or* gates and *buf/not* gates.

- And/Or Gates

  - and or xor
  - nand nor xnor

- Buf/Not Gates

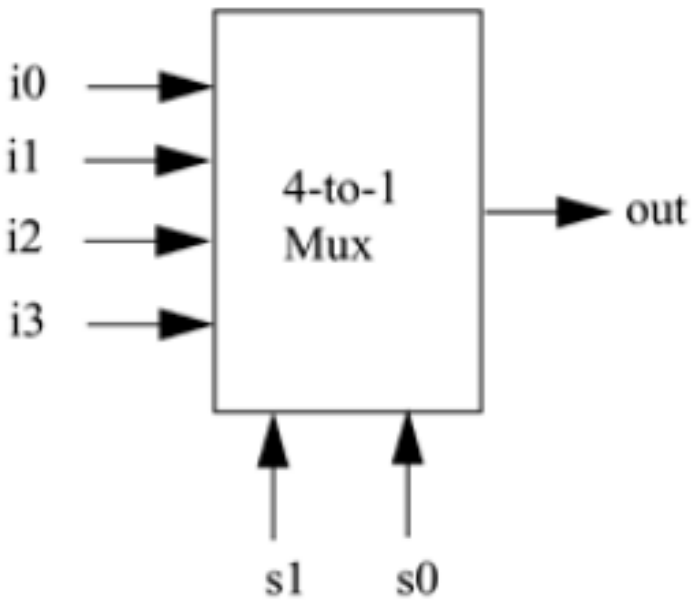
  - buf not

Example: Instantiation of And/Or gates

```
wire OUT, IN1, IN2;
// basic gate instantiations.
and a1(OUT, IN1, IN2);
nand na1(OUT, IN1, IN2);
or or1(OUT, IN1, IN2);
nor nor1(OUT, IN1, IN2);
xor x1(OUT, IN1, IN2);
xnor nx1(OUT, IN1, IN2);
// More than two inputs; 3 input nand gate
nand na1_3inp(OUT, IN1, IN2, IN3);
// gate instantiation without instance name
and (OUT, IN1, IN2); // legal gate instantiation
```

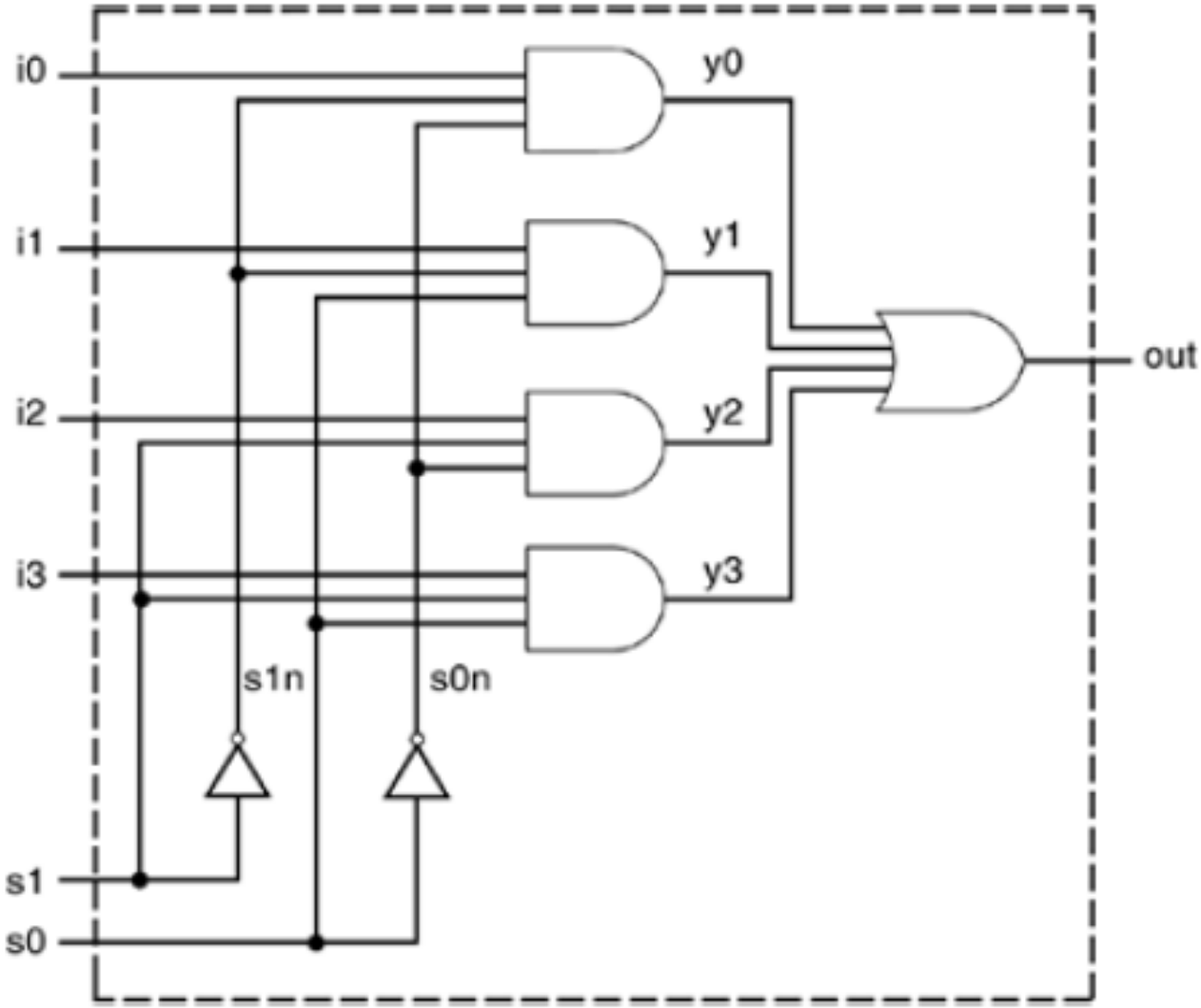
Example 1:

- 4x1 multiplexer



s1	s0	out
0	0	I0
0	1	I1
1	0	I2
1	1	I3

Example 1: (Contd.)



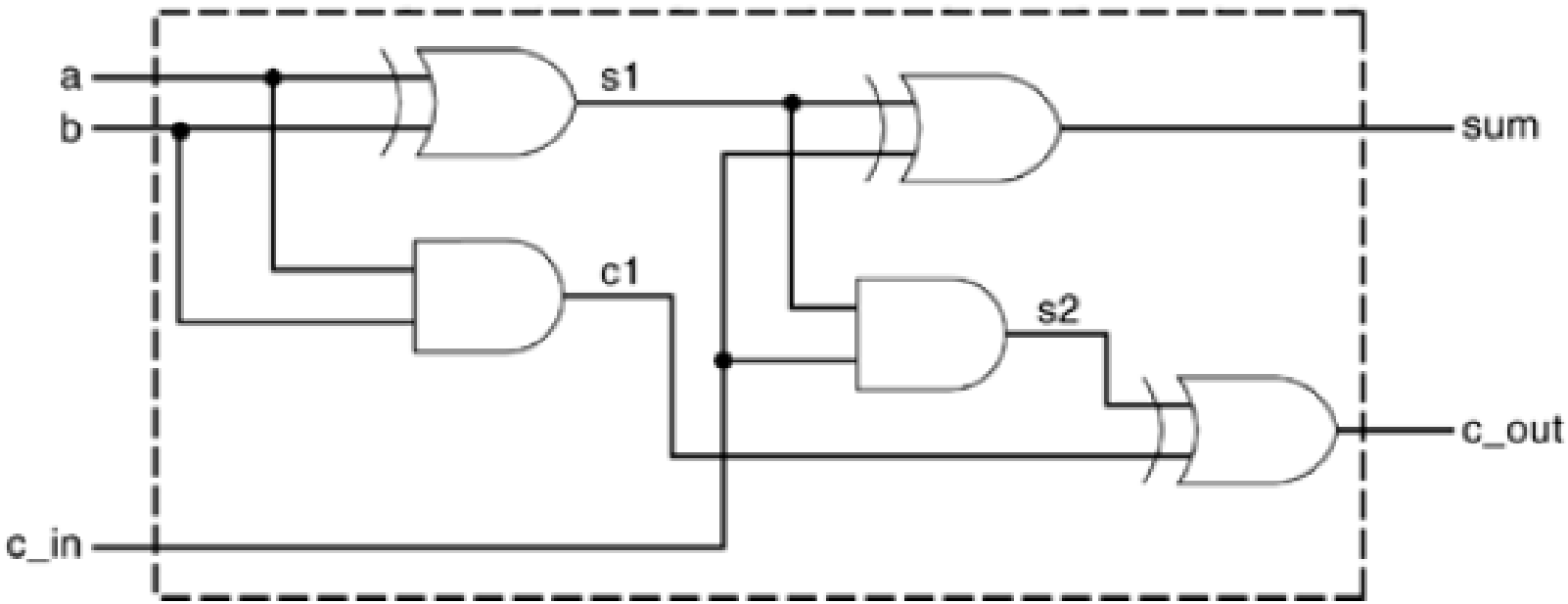
Example 1: (Contd.)

```
// 4-to-1 multiplexer.
module mux4_to_1 (out, i0, i1, i2, i3, s1, s0);
// Port declarations from the I/O diagram
output out;
input i0, i1, i2, i3;
input s1, s0;

// Internal wire declarations
wire s1n, s0n;
wire y0, y1, y2, y3;

// Gate instantiations
// Create s1n and s0n signals.
not (s1n, s1);
not (s0n, s0);
// 3-input and gates instantiated
and (y0, i0, s1n, s0n);
and (y1, i1, s1n, s0);
and (y2, i2, s1, s0n);
and (y3, i3, s1, s0);
// 4-input or gate instantiated
or (out, y0, y1, y2, y3);
endmodule
```

Example 1: (Contd.)

```
// Define the stimulus module (no ports)
module stimulus;

// Declare variables to be connected
// to inputs
reg IN0, IN1, IN2, IN3;
reg S1, S0;
// Declare output wire
wire OUTPUT;

// Instantiate the multiplexer
mux4_to_1 mymux(OUTPUT, IN0, IN1, IN2, IN3, S1, S0);

// Stimulate the inputs
initial
begin
    // set input lines
    IN0 = 1; IN1 = 0; IN2 = 1; IN3 = 0;
    #1 $display("IN0= %b, IN1= %b, IN2= %b, IN3= %b\n",IN0,IN1,IN2,IN3);
    // choose IN0
    S1 = 0; S0 = 0;
    #1 $display("S1 = %b, S0 = %b, OUTPUT = %b \n", S1, S0, OUTPUT);
    // choose IN1
    S1 = 0; S0 = 1;
    #1 $display("S1 = %b, S0 = %b, OUTPUT = %b \n", S1, S0, OUTPUT);
    // choose IN2
    S1 = 1; S0 = 0;
    #1 $display("S1 = %b, S0 = %b, OUTPUT = %b \n", S1, S0, OUTPUT);
    // choose IN3
    S1 = 1; S0 = 1;
    #1 $display("S1 = %b, S0 = %b, OUTPUT = %b \n", S1, S0, OUTPUT);
end
endmodule
```

4-bit Ripple Carry Full Adder

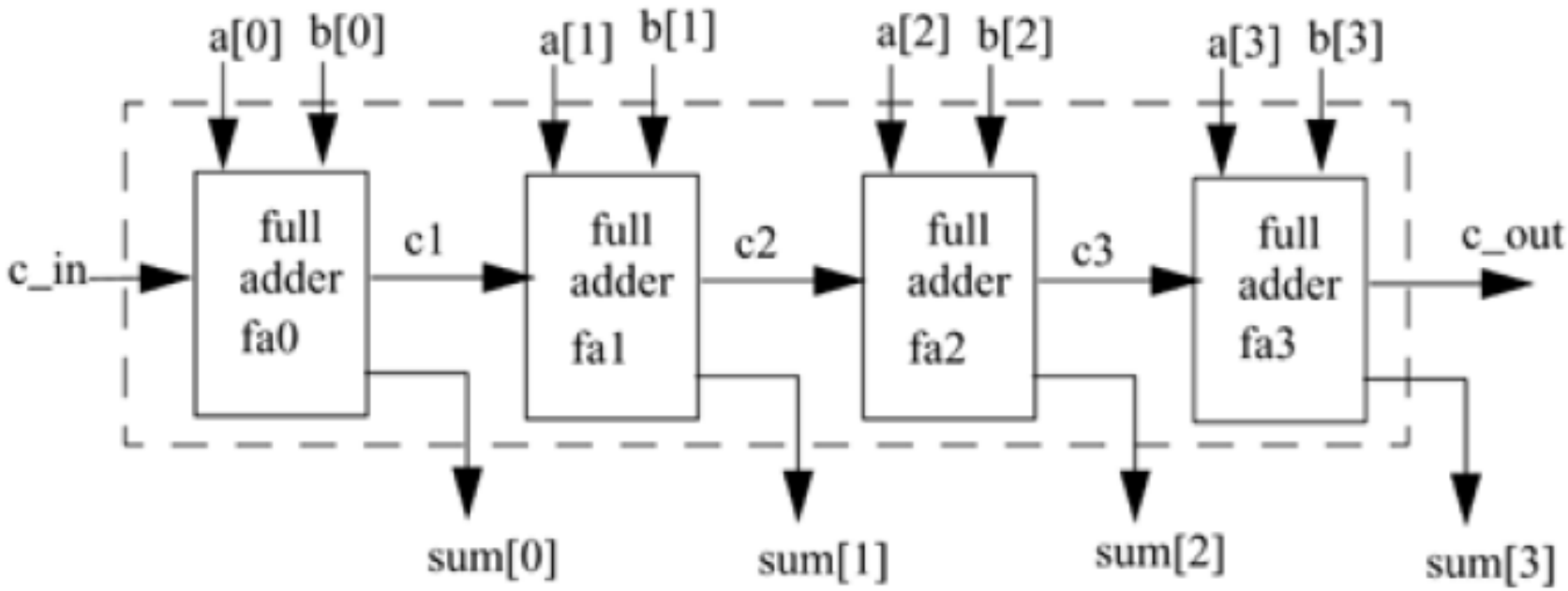


1-bit Full Adder

4-bit Ripple Carry Full Adder (Contd.)

```
// Define a 1-bit full adder
module fulladd(sum, c_out, a, b, c_in);
// I/O port declarations
output sum, c_out;
input a, b, c_in;
// Internal nets
wire s1, c1, c2;
// Instantiate logic gate primitives
xor  (s1, a, b);
and (c1, a, b);
xor (sum, s1, c_in);
and (c2, s1, c_in);
or  (c_out, c2, c1);
endmodule
```

4-bit Ripple Carry Full Adder (Contd.)



4-bit Ripple Carry Full Adder (Contd.)

```
// Define a 4-bit full adder
module fulladd4(sum, c_out, a, b, c_in);

// I/O port declarations
output [3:0] sum;
output c_out;
input[3:0] a, b;
input c_in;

// Internal nets
wire c1, c2, c3;

// Instantiate four 1-bit full adders.
fulladd fa0(sum[0], c1, a[0], b[0], c_in);
fulladd fa1(sum[1], c2, a[1], b[1], c1);
fulladd fa2(sum[2], c3, a[2], b[2], c2);
fulladd fa3(sum[3], c_out, a[3], b[3], c3);

endmodule
```

4-bit Ripple Carry Full Adder (Contd.)

```
// Define the stimulus (top level module)
module stimulus;

// Set up variables
reg [3:0] A, B;
reg C_IN;
wire [3:0] SUM;
wire C_OUT;

// Instantiate the 4-bit full adder. call it FA1_4
fulladd4 FA1_4(SUM, C_OUT, A, B, C_IN);

// Setup the monitoring for the signal values
initial
begin
    $monitor($time," A= %b, B=%b, C_IN= %b,, C_OUT= %b, SUM= %b\n",
                                                A, B, C_IN, C_OUT, SUM);
end
// Stimulate inputs
initial
begin
    A = 4'd0; B = 4'd0; C_IN = 1'b0;

    #50 A = 4'd3; B = 4'd4;

    #50 A = 4'd2; B = 4'd5;

    #50 A = 4'd9; B = 4'd9;

    #50 A = 4'd10; B = 4'd15;

    #50 A = 4'd10; B = 4'd5; C_IN = 1'b1;
end
```

References

- Chapter 1-5, Verilog HDL by Samir Palnitkar, Second Edition.

Thank you